CareSync

A Full-Stack Clinic Appointment & Patient Management System with AI-Assisted Scheduling, Deployed on Microsoft Azure

---

1. Project Overview

CareSync is a role-based web application for the operational side of a clinic. Patients search for doctors and book genuinely open slots; doctors run their day from one dashboard and open full charts for patients they have treated; admins see live clinic analytics and manage doctors, patients, departments and appointments. A small set of administrative AI features estimate the likelihood that a booking will be missed and categorise the type of visit requested.

CareSync is explicitly **not** a diagnostic system. It never suggests treatment or makes a clinical judgement; every AI feature operates on scheduling data only. All data is synthetic, and the system is not intended for real clinical use.

It targets problems that small clinics typically manage with phone calls and spreadsheets:

- **Double-booking and scheduling conflicts** when there is no single system of record.
- **No-shows** that waste a slot another patient could have used, with no way to anticipate them.
- **Fragmented patient history** - notes, prescriptions and past visits scattered rather than attached to a record.
- **No operational visibility** into volume, cancellations or which departments are busiest.

2. Project Objectives

- Let patients self-serve: search doctors, see real open slots, and book, reschedule or cancel without a phone call.
- Give doctors one daily workspace for status, consultation notes and prescriptions, plus a full patient chart for every patient they have treated.
- Give admins a live dashboard (volumes, status mix, department load, doctor utilisation) and management screens for the whole clinic.
- Use lightweight, honestly-scoped ML to estimate no-show risk and tag visit types - administrative assistance, never diagnosis.
- Enforce access control for real: patients see only their own data, and a doctor can open a patient's chart only if they have an actual appointment with that patient.
- Build and ship it like a small production system: typed contracts, database-level constraints, automated tests on both tiers, CI, containers and a repeatable cloud deployment.

3. Motivation

The goal was a project closer to a real internal tool than a typical CRUD exercise: a domain with genuine constraints (conflicting bookings, role boundaries, who may see what) where each constraint is actually enforced in code and covered by a test rather than only described in a README.

A second motivation was honesty about where AI helps. Categorisation is a transparent rule-based classifier because it is a small closed set that benefits from being auditable; no-show risk uses a trained model because the relationship between a patient's history and their likelihood of missing a visit is not something a fixed rule captures well. That distinction is stated in the product itself.

4. Key Features

Patient

- Dashboard with next appointment, counts and recent activity; search doctors by name or department.
- Booking flow with a date strip (days a doctor does not work are disabled) and morning / afternoon / evening slot groups; past times today are hidden.
- Book, reschedule (same slot picker) and cancel (with confirmation); upcoming / past / cancelled views.
- Prescriptions (printable) and medical records; editable profile with a completeness meter; change password; in-app notifications with mark-all-read.

Doctor

- Daily dashboard: still to see, completed, high no-show risk, patients under care; today's schedule and what is coming up.
- A single consultation workspace: confirm / complete / no-show / cancel, consultation notes, and an itemised prescription builder with frequency presets.
- All-appointments view with search and upcoming / past / cancelled tabs; patient list with a chart drawer (overview, visits, records, prescriptions, add a chart note).
- Visual weekly availability editor: add the same block to several days at once, overlap detection, weekly-hours summary.

Admin

- Analytics dashboard: eight KPIs, a gap-free 14-day trend, status donut, appointments by department, doctor utilisation and recent bookings.
- Appointments console with search, status filter and inline status change; doctors (add, edit, remove with confirmation); patients with activity counts; departments.

AI (administrative only)

- **No-show risk** on every booking, computed from the patient's own history and shown to the doctor as Low / Medium / High.
- **Visit categorisation** into General consultation, Follow-up, Routine check-up or Specialist consultation.
- **Scheduling insights** endpoint: peak hours, busiest departments, cancellation and no-show rates, utilisation.

5. Technology Stack

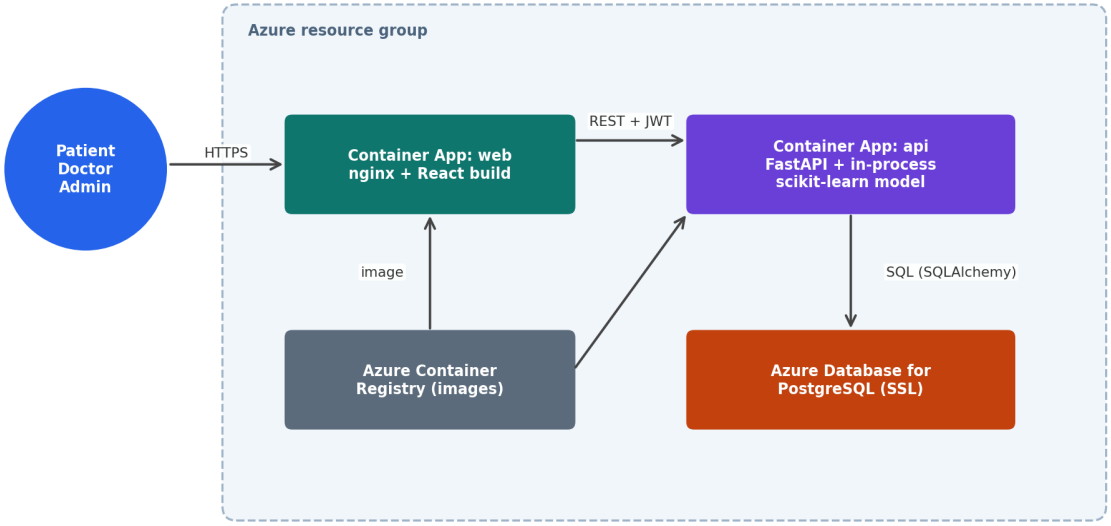
The stack matches the system's real size: one backend process serves the API and runs the model in-process, with no queue, cache layer or separate ML service because nothing about this scale needs one.

Technology	Purpose
React 18 + Vite	Single-page frontend; Vite for fast development and a small, chunk-split production build.
React Router	Role-gated route trees for patient, doctor and admin.
Hand-built CSS design system	Design tokens (colour, type, spacing) and components written in plain CSS - no utility framework.
Recharts, lucide-react	Admin charts and the icon set.

Technology	Purpose
axios	HTTP client with JWT and 401 interceptors; backend address resolved at runtime.
FastAPI + Pydantic v2	REST API with automatic OpenAPI docs; every request and response is typed and validated.
SQLAlchemy 2.0	Ten relational models with foreign keys, a unique constraint and indexes.
python-jose, passlib/bcrypt	JWT issuing and verification; password hashing.
PostgreSQL	Primary database (SQLite supported for quick local runs and tests).
scikit-learn, pandas, NumPy	No-show model: RandomForestClassifier trained on a synthetic dataset.
pytest + httpx	28 backend tests: auth, RBAC, booking conflicts, availability, AI, chart access, password change.
Vitest + Testing Library	20 frontend tests rendering every page for every role, plus route-guard checks.
Docker, Docker Compose, nginx	Reproducible images; nginx serves the SPA and writes runtime config at container start.
GitHub Actions	CI on every push: both test suites, frontend build, both Docker builds.
Azure Container Apps, ACR, Azure Database for PostgreSQL	Cloud hosting. Images are built in Azure; PostgreSQL Flexible Server with SSL required.

6. System Architecture

CareSync is a three-tier system: a React single-page app served by nginx, a FastAPI backend, and PostgreSQL. The browser never talks to the database; every read and write goes through the API, which enforces role-based access before touching data. The AI model runs **in-process** in the API - a deliberate trade-off, since the model is small enough to load once and call synchronously, and a separate service would add a network hop and a second deployable for no benefit at this scale.



API address is injected into the web container at start (config.js); CORS on the API allows only the web origin.

Fig. 1 - Deployment architecture on Azure. Images are built in Azure Container Registry; secrets are Container Apps secrets referenced by name.

7. Project Workflow

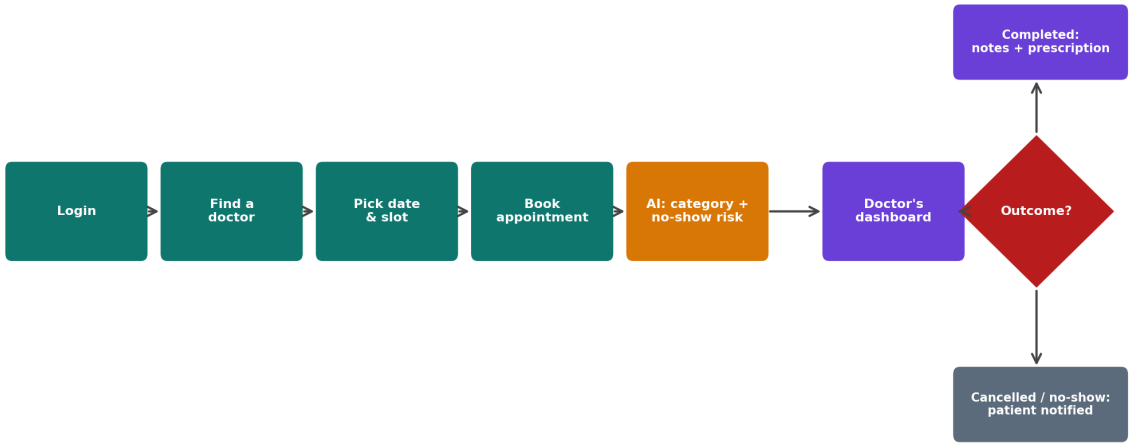


Fig. 2 - End-to-end booking workflow from login to the doctor's disposition of the visit.

- **Login:** the user authenticates and receives a role-scoped JWT.
- **Find a doctor:** filter by department or search by name or specialisation.
- **Pick a slot:** the API expands the doctor's weekly availability for the chosen date and removes booked times, so only genuinely open slots appear.
- **Book:** a unique database constraint on (doctor, date, time) rejects a double-booking even under a race; the API returns a clean 409.
- **AI step:** the booking is categorised and scored for no-show risk before it is saved.
- **Doctor's dashboard:** the booking appears with its risk badge; the doctor opens the consultation workspace during the visit.
- **Outcome:** completed visits get notes and optionally a prescription; cancelled or no-show visits notify the patient.

8. Machine Learning Pipeline - No-show Prediction



Training is offline (ml/train_no_show_model.py). The API loads the saved model once and calls predict_proba() on each booking.

Fig. 3 - Offline training versus the online prediction made on each booking.

Data.

6,000 synthetic historical appointments are generated with a deliberately noisy logistic relationship between features and the label, so the model is not trivially perfect. Features: patient age, prior appointments, prior no-shows, prior cancellations, booking lead time, day of week and appointment hour.

Split and model.

A stratified 80/20 train/test split keeps the no-show rate equal in both sets. The model is a RandomForestClassifier (200 trees, max depth 8, balanced class weights), chosen for robustness on mixed-scale tabular data, resistance to overfitting and built-in feature importances, with no scaling step required.

Evaluation (synthetic test data; varies slightly per retraining):

Metric	Score
Accuracy	about 0.67
Precision	about 0.43
Recall	about 0.48
F1-score	about 0.46

These numbers are modest on purpose: the data is noisy and synthetic, and the score is a scheduling aid, not a clinical measure. Lead time, prior no-shows and age were the strongest predictors.

Serving.

The trained model is saved with joblib, committed to the repository so every build ships it, and loaded once at API start. Each booking computes its own feature vector from the patient's real history and calls predict_proba() synchronously. If the model file were ever missing, the API falls back to a transparent heuristic rather than failing. The API response states that the estimate is administrative and not medically validated.

9. Database Design

Ten PostgreSQL tables with real foreign keys, one database-level unique constraint, and indexes on foreign keys and commonly filtered columns (status, date, role, specialisation).

Table	Purpose	Key relationships
users	Identity and authentication for all three roles.	1:1 to patients or doctors

Table	Purpose	Key relationships
patients	Profile: date of birth, gender, blood group, address.	N:1 to users
doctors	Profile: specialisation, licence, fee, bio.	N:1 to users, departments
departments	Groups doctors for filtering and analytics.	1:N to doctors
doctor_availability	Recurring weekly time blocks.	N:1 to doctors
appointments	Central transactional table, incl. category and no-show risk.	N:1 to patients, doctors
medical_records	Chart notes, optionally tied to a visit.	N:1 to patients
prescriptions	One per appointment.	N:1 to appointments
prescription_items	One row per medicine.	N:1 to prescriptions
notifications	Bookings, status changes, prescriptions.	N:1 to users

The unique constraint on (doctor_id, appointment_date, appointment_time) means a double-booking fails at the database even if two requests race; the API converts the integrity error into a 409 Conflict.

10. AI & Data Science Components

- **No-show prediction** - the only genuinely trained model, because a patient's history is a richer signal than any fixed rule.
- **Appointment categorisation** - transparent keyword rules, chosen over a black-box model so staff can see why a booking was tagged.
- **Scheduling insights** - SQL aggregation over a rolling 90-day window.
- **Risk shown where decisions are made** - on the doctor's appointment row and in the new-booking notification, not buried in a report.

11. Challenges Faced & Solutions

These are real problems hit and fixed while building and testing the system.

Build-time versus runtime configuration

Challenge: Vite bakes VITE_API_URL into the bundle at build time, but it was supplied as a Docker runtime variable. The frontend silently fell back to a relative path, which the static server answered with an HTML page and a misleading 200 status, so a 'successful' login returned no user and redirected to /undefined.

Solution: Reproduced the bug deliberately, then redesigned configuration: the container writes config.js from an API_URL variable at start, so one image works in any environment with no rebuild. This also underpins the Azure deployment.

A login schema that locked out the seeded admin

Challenge: The login request schema validated email format, so the seeded admin (a .local address) was rejected with a 422 before the password was even checked - and the same strict validation on response models could turn stored data into 500 errors. A regression test written for a related bug exposed it.

Solution: Login now only looks the account up; output schemas return stored emails as plain strings; the default admin moved to a valid domain. A test covers a legacy .local account.

An AI feature that was built but never ran

Challenge: The model, endpoint and UI all existed, yet nothing called the model at booking time, so the risk score stayed null forever.

Solution: Wired prediction into the booking endpoint using the patient's real history and added a test asserting the score is populated.

Authorization looser than intended

Challenge: Any doctor could open any patient's profile, records and prescriptions by ID, and could list every patient.

Solution: Chart access now requires an actual appointment between that doctor and patient; the all-patients list is admin-only. Tests cover both directions.

A well-known admin password on a public deployment

Challenge: Seeding hard-coded a documented admin password - acceptable locally, dangerous on a public URL.

Solution: Seed credentials come from environment variables; the Azure script generates random ones and stores them as secrets; the production build refuses to start with the default JWT key; passwords are never printed to logs.

An interface that did not look professional

Challenge: A theme tied to the browser's colour scheme produced a dark page, a pale sidebar and bright form fields at once, with raw date pickers and no empty or loading states.

Solution: Replaced it with a single design system (navy navigation, white surfaces, one accent, semantic status colours), real icons, a slot-picker, modals and toasts, and skeleton / empty states throughout. Twenty page-render tests keep it from regressing.

12. Deployment on Azure

A single script (PowerShell or Bash) provisions and deploys everything with the Azure CLI: a resource group, Container Registry, PostgreSQL Flexible Server, a Container Apps environment and two container apps. Images are built in Azure, so no local Docker is needed. It generates strong secrets, stores them as Container Apps secrets, configures CORS between the two apps, waits for the health check, and prints the URLs and demo logins. Re-running it rolls out a new version; a companion script deletes the resource group to stop charges.

Verification status. The backend, the frontend, and the end-to-end patient, doctor and admin journey were executed and passed (including production mode and startup seeding). The Azure scripts themselves could not be run against a live subscription in the authoring environment; the Bash script was syntax-checked and the logic was reviewed, but the first real deployment may need a region or flag adjustment.

13. Future Enhancements

- Alembic migrations in place of create-on-start; private networking and Key Vault for the cloud deployment; managed-identity image pulls.
- Refresh tokens and login rate limiting; pagination on list endpoints; editing and deleting medical records.
- Email or SMS reminders (the notifications table already supports them); a trained text classifier for categorisation once real labelled data exists.
- Multi-clinic support; end-to-end browser tests (Playwright) to complement the render tests.

14. Learning Outcomes

- **Full-stack ownership** across schema, API contract and UI, kept consistent as the system grew.
- **Applied ML with honest scope:** training, evaluating and actually wiring a model into a live request path, and choosing rules where rules are more defensible.
- **Authorization as a design problem:** the most important fixes were permission boundaries that were too loose, not missing features.
- **Debugging across build and deploy boundaries:** several bugs were about when code runs (build time, startup, async races), invisible in local development.
- **Tests that encode real bugs**, so regressions fail loudly.
- **Cloud delivery:** container images, runtime versus build configuration, managed PostgreSQL with SSL, secrets handling and repeatable deployment scripts.

15. Conclusion

CareSync demonstrates full-stack engineering applied to a real domain: role-based access enforced end to end, a relational schema with database-level guarantees, a model that is trained, evaluated and genuinely used in the request path, a professional interface backed by automated tests, and a repeatable cloud deployment.

It does not claim to be more than it is. It is a portfolio-grade system built on production patterns, not a production system: there are no migrations, modest test depth for a live clinical tool, no rate limiting, and the cloud deployment is documented and scripted but still needs its first live run. Stating those limits plainly is part of engineering it honestly.